

as *Person, Company, Location, Month, Day, Book*, etc. If the above example is tagged with entities, it will be tagged as <Person> Sandya </Person>, <Movie>Star Wars</Movie>, <Book> Pride and Prejudice </Book>, <Location> Cubbon Park </Location> , etc.

It is not only important that the NER system recognises a phrase correctly as an entity but also that it labels it with the right entity type. Consider the sentence, “Washington Jr went to school in England, but for graduate studies, he moved to the United States and studied at Washington.” This sentence contains two references to the noun ‘Washington’, one as a person: ‘Washington Jr’ and another as a location: ‘Washington, United States’. While it may appear that if an NER system has a list of all pronouns, it can correctly extract all entities, in reality, this is not true. Consider the two sentences, “Jobs are hard to find...” and “Jobs said that the employment rate is picking up...” Even if the NER system has an exhaustive list of pronouns, it needs to figure out that the word ‘Jobs’ appearing in the first sentence does not refer to an entity, whereas the reference ‘Jobs’ in the second sentence is an entity.

Given our discussion so far, it is clear to us that NER systems can be built in a number of ways, though no single method can be considered to be superior to others and a combination of techniques is needed. We saw that rule-based NER systems tend to be incomplete and have the disadvantage of requiring manual extension quite frequently. Rule-based systems use typical pattern matching techniques to identify the entities. On the other hand, it is possible to extract features associated with named entities and use them to train classifiers that can tag entities, using machine learning techniques. Machine learning approaches for identifying entities can be based on: (a) supervised learning techniques; (b) semi-supervised learning techniques; and (c) unsupervised learning techniques.

The third kind of NER systems can be based on gazetteers, wherein a lexicon or gazette for names is constructed and made available to the NER system which then tags the text, identifying entities in the text based on the lexicon entries. Once a gazetteer is available, all that the NER needs to do is to have an efficient lookup in the gazetteer for each phrase it identifies in the text, and tag it based on the information it finds in the gazette. A gazette can also help to embed external world information, which can help in name entity resolution. But first, the gazette needs to be built for it to be available to the NER system. Building a gazette can consume considerable manual effort. One of the alternatives is to build the lexicon or gazetteer itself through automatic means, which brings us back to the problem of recognising named entities automatically from various document sources. Typically, external world sources such as Wikipedia or Twitter can be used as the information sources from which the gazette can be built. Sometimes a combination of approaches can be used with a lexicon, in conjunction with a rules-based or machine learning approach.

While rule-based NER systems and gazetteer approaches work well for a domain-specific NER, machine learning approaches generally perform well when applied across multiple domains. Many of the machine learning based approaches use supervised learning techniques, by which a large corpus of text is annotated manually with named entities and the goal is to use the annotated data to train the learner. These systems use statistical models and some form of feature identification to make predictions about named entities in unlabelled text, based on what they have learnt from the annotated text. Typically, supervised learning systems study the features of positive and negative examples, which have been tagged as named entities in the hand-annotated training set. They use that information to either come up with statistical models, which can predict whether a newly encountered phrase is a named entity or not. If it is a named entity, supervised learning systems predict its type as well. In the next column, we will continue our discussion on how hidden Markov models and maximum entropy models can be used to construct learner systems.

My ‘must-read book’ for this month

This month’s book suggestion comes from one of our readers, Jayshankar, and his recommendation is very appropriate for this month’s column. He recommends an excellent resource for text mining—a book called ‘Taming Text’ by Ingwersol, Morton and Farris. The book describes different algorithms for text search, text clustering and classification. There is also a detailed chapter on Named Entity Recognition, which will be useful supplementary reading for this month’s column. Thank you, Jay, for sharing this book link.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book’s name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions or software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya’s LinkedIn group ‘Computer Science Interview Training India’ at <http://www.linkedin.com/groups?home=HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20HYPERLINK%20http://www.linkedin.com/groups?home=&gid=2339182%20gid=2339182>